Week-3

Cuda WiDS

Assumptions made/points to be noted:

1) From submission only one kernel seems to be needed.

2) I will be using the elemwise multiply and scale kernel here.

3) coalesced.cu, non_coalesced.cu, shared_memory.cu and cpu_baseline.py are the files created.

Results:

coalesced.cu which also uses global memory was used as the baseline for comparsion.

coalesced.cu

kernel execution time: 0.005952 ms

No of correct: 5605/100000

non-coalesced.cu

Kernel execution time: 0.011328 ms

No of correct: 5453/100000

shared_memory.cu

Kernel execution time: 0.026496 ms

No of correct: All correct.

Coalesced access is nearly 2 times faster than non-coalesced access.

This observation can be easily correlated to the fact that in coalesced access memory allocation for threads in a warp are next to each other, while strided access leads to memory for a single warp being separated in memory space. And noting that memory is allocated to a warp at a time and not a thread at a time.

Shared_memory is nearly 4 times slow when compared to gloabal usage(coalesced.cu), this is majorly because of the kernel I have used. Shared memory is useful when threads reuse the stored memory but as far as elem wise multiply and scale is concerned the memory is used once and then we are done here.

On a side note, such kernels which use data once and yet are used often like ReLU are often fused with other kernels to make use of shared memory optimally. I came across this when learning about shared memory hence thought I would share.

Thus, shared memory does not speed up the given kernel.

Banking conflicts do not exist in this code(analytically), since every thread of a warp as far as my code is concerned access in a coalesced way ensuring that 0-31 are accessed (threadIdx.x). Bank is thread modulo 32 thus bank = thread.